Who Writes the Code That Trains the Models? AI Co-Authorship Trailers in AI Infrastructure Repositories, 2023–2026

Thanatchon Chongmankhong

October 2026

Abstract Coding agents now sign their work: Claude Code, Copilot's cloud agent, Codex, Cursor, Jules and others leave co-author trailers, agent author accounts or verbatim markers in commit metadata. We use these signatures to measure, month by month from 2023 to 2026, the share of commits to the open-source repositories that train, fine-tune, evaluate and serve AI models that carry an explicit AI attribution, and compare it with matched popular non-AI repositories and with the coding-agent harnesses themselves. The design was pre-registered before any study repository was scanned. Every repository's full default-branch history was read on one day (96 repositories, 2,850,951 commits), and matched against a published catalogue of 52 signatures for 28 tools, each backed by three public example commits. In July–September 2026, 8.7% of commits to AI-infrastructure repositories were AI-attributed (95% CI 4.0%–14.5%), up from 0.5% over 2025 and almost none in 2024. Matched non-AI repositories reached 7.1% and the coding-agent harnesses 10.3%, with intervals too wide to tell the three groups apart: we find no evidence that AI infrastructure is ahead, but cannot rule out a sizeable lead either. Of the pre-registered hypotheses about level, difference, growth, recursion and tools, only the level hypothesis meets its decision rule, and only on its point estimate; its interval reaches below the threshold. Attribution is a lower bound: it can be switched off, lost at merge, dropped when code is exported from a private monorepo, or never added. We measure the merge path directly: 92% of sampled attributed pull requests keep an attribution on the default branch. We release the scanner so that any organisation can draw the same curve for its own repositories.

1 Introduction

Frontier laboratories now say that most of their own code is written with their models. Anthropic reports that more than eighty percent of the lines of code merged into its codebase in May 2026 could be attributed to Claude [1]. Such figures come from inside one laboratory, count lines rather than commits, and cannot be checked from outside. The question behind them can be asked in public, though: *how much of the open-source code that trains and serves AI models is already produced with AI tools?* We answer a narrower and checkable version of it: what share of commits to that code *declare* AI involvement in their metadata?

We find that the answer in mid-2026 is about one commit in 11 (8.7% in July–September 2026), and almost nothing before 2025 (Figure 1). The comparison is the part we expected to be different. Popular repositories with nothing to do with AI, matched on stars, language and activity, carry AI attribution at a similar rate (7.1%), and in 6 of the 9 months of 2026 the controls were *above* the AI-infrastructure repositories. The open-source coding-agent harnesses, the tools that produce these attributions, started earlier but are not clearly ahead in 2026. We read this as inconclusive rather than as equality: the intervals are wide, one control repository supplies 54% of the controls' attributed commits in the quarter, and some AI-infrastructure repositories are exported from internal monorepos, so their public commits cannot show attribution. What the data do not show is the clear lead of "AI building AI" that we pre-registered.

The instrument is simple, and the simplicity is deliberate. Many coding agents add a signature to the commits they help write: Claude Code adds a `Co-Authored-By` trailer by default [2], GitHub's cloud agent authors commits under its own account [3], and Cursor, Jules, Aider and Codex have their own conventions [4], [5], [6], [7]. Counting these signatures is not new [8], [9], [10]. What this study adds is (i) a pre-registered monthly series for a defined population of AI-infrastructure repositories, with matched controls and the agent harnesses as a third group; (ii) a catalogue in

which every signature is backed by three public example commits; (iii) a direct measurement of how often attribution on a pull request's commits survives the merge to the default branch; and (iv) a link between the measured share and each repository's stated AI-contribution policy. The scanner, `coauthored`, is a small open-source command-line tool; it reproduces every number in this paper from a recorded snapshot and draws the same curve for any organisation's repositories.

Throughout, the unit is the **AI-attributed commit**, and every share is a **lower bound** on AI involvement. An attributed commit is not a commit "written by AI": the trailer says a tool took part, not how much of the change it produced. Attribution is also not adoption. It measures which tools sign and which repositories keep the signature, not who uses AI: the repository of OpenAI's Codex harness had 1 attributed commit of 3,735 in 2026-Q3. Section 3 explains how attribution goes missing.

2 Related work

Measuring AI's share of code. Daniotti et al. train a classifier to detect AI-written Python functions in GitHub commits and estimate national shares and productivity effects [11]. Their detector measures code style; we count declarations. The two quantities have different errors and should not be compared one-to-one. Horikawa et al. find that once signatures are masked, detectors of agent commits lose much of their precision at realistic base rates, which suggests that signature-based studies like ours report a lower bound [9].

Counting signatures. Robbes et al. estimate project-level coding-agent adoption across a large GitHub sample from several signals, among them co-author trailers, agent accounts, configuration files, branch names and labels [8]; their unit is the project, ours the commit. AIDev collects agent-authored pull requests from five agents [12]. Blog-scale and dashboard counts of `Co-Authored-By: Claude` commits across public GitHub [10], [13], [14] and GitHub's own telemetry on agent-started pull requests [15] report fast growth; several describe their counts as incomplete, and none fixes a population, compares it with controls, or measures loss at merge. Our contribution is a narrow, pre-registered and reproducible series rather than a new signal.

Bots, trailers and merges. Separating automation from people in commit history has a long history in mining software repositories [16]. Our denominator removes maintenance bots (dependency updates, releases) but keeps AI-agent accounts, which do commit code on someone's behalf. The `Co-authored-by` convention [17] interacts with squash merging: a repository chooses whether the squash message holds the pull request's title, its description or a list of its commits [18], and GitHub credits every commit author as a co-author of the squash commit [19].

Policies. Open-source projects have adopted AI-contribution policies ranging from outright bans [20], [21], [22] to disclosure requirements [23], [24] and the Linux kernel's `Assisted-by:` tag [25]. Hora and Robbes classify the policies of popular repositories [26]; we relate a similar classification to measured attribution.

Productivity and usage. Experiments find that AI assistance changes the speed and volume of programming work, in both directions [27], [28], [29], that it raises open-source contributions [30], and that agentic tools are used mostly to delegate whole tasks [31]. Task-length benchmarks show fast capability growth [32]. None of these measures what share of a real repository's history declares AI involvement.

3 What an attribution is, and how it goes missing

A commit is **AI-attributed** if (a) a `Co-authored-by:`, `Assisted-by:` or `Co-developed-by:` line in its message names an identity in the signature catalogue, (b) its author or committer is an AI-agent account in the catalogue, or (c) it carries a tool's verbatim marker such as `Generated with Claude Code`. Identities are matched in full on e-mail address and/or name; no free-text keyword ("AI", "GPT", "Claude") counts. A `Signed-off-by:` line never counts, because it is a human's certification of origin. The Linux kernel's `Assisted-by: AGENT:MODEL` format carries no e-mail address and cannot be tied to a catalogue identity; we report it as a separate indicator (140 commits in S1 and 62 in S2 over the window, none in S3).

The signal is a lower bound for four reasons.

1. **The trailer is switched off.** Every major tool lets the user disable or change attribution [2], [4], [5], [7]. Since February 2026 Jules can commit as the user alone [5], which leaves no trace at all.
2. **The trailer is stripped at merge.** A trailer on a pull request's commits reaches the default branch only if the merge keeps it: a merge commit and a rebase keep the original commits, but a squash commit carries whatever message the repository's squash setting and the person merging produce [18]. We measure this loss in Section 6.

3. **The commit is exported from elsewhere.** Some projects are developed in a company's internal repository and exported to GitHub by a tool that writes its own commit (in S1, for example, `openxla/xla`, `jax-ml/jax` and `tensorflow/tensorflow`). Whatever attribution the internal change had is not visible. Removing the repositories with mirror-export trailers raises the S1 share in 2026-Q3 to 12.2% (6.6%–18.5%).
4. **The tool never attributes.** Autocomplete, chat assistants and copy-paste leave nothing in git, and some agents add no signature by default. Some policies also exempt maintainers from disclosure [24].

The tool mix in the data is therefore partly a property of the instrument: tools that sign by default are over-represented relative to their use. Pre-registered hypothesis H5 tests the instrument as much as the market.

4 Pre-registered design

The design, the repository lists, the bot list and the signature catalogue were frozen in a pre-registration tagged `prereg-v1` (commit `27a6847`) before any study repository was scanned; three pilot repositories, excluded from every analysis, were used to debug the scanner and discover signatures. Every later clarification is logged with its effect (`DEVIATIONS.md`).

Strata. Inclusion rules were applied to GitHub metadata before scanning. All repositories are public, not archived and not forks, with at least two thousand stars and at least two hundred default-branch commits in the twelve months to October 2026; S1 and S3 repositories also had at least a hundred commits in 2024, so that the series has a pre-agent baseline. **S1** (42 repositories) contains model-building infrastructure across seven layers: training frameworks (PyTorch, JAX, Keras), model libraries (Transformers, PEFT, TRL, Diffusers), distributed training (DeepSpeed, Megatron-LM), fine-tuning and reinforcement learning, inference and serving (vLLM, SGLang, llama.cpp, Ollama), kernels and compilers (Triton, FlashAttention), and evaluation and data. **S2** (12) contains open-source coding-agent harnesses whose primary product is a coding agent (e.g. OpenHands, Cline, Gemini CLI, Codex, Qwen Code). **S3** (42) contains one matched control per S1 repository, drawn from popular non-AI repositories by a seeded nearest-neighbour match on stars and recent commit activity within the same language group (caliper ±50%), then screened so that no control has model training or inference at its core. The lists are in the appendix (Table 2); every exclusion and its reason is in `configs/repos.yaml` and `configs/s3_screen.yaml`.

Snapshot. All 96 repositories were cloned on one UTC calendar day, 2026-10-28, with commit objects only, and their full default-branch history was scanned (2,850,951 commits, 631 seconds of clone and scan time, 1.32 GB). Head SHAs are recorded, and all counting is from those SHAs. Author and committer identities are replaced by a salted hash at ingest; no names, e-mail addresses or messages are stored.

Counting. The primary unit is a non-merge commit reachable from the default branch, dated by its committer date in UTC, in the window January 2023 to September 2026. Commits authored by maintenance bots are removed from the denominator; AI-agent accounts stay. Shared history (a fork or a vendored copy) is counted once per stratum, in the earliest-created repository. The catalogue has 52 entries for 28 tools (31 co-author identities, 16 agent accounts, 5 markers), each with three public commits as evidence and a passing and a failing test case. On a labelled sample of 200 pilot commits (80 attributed), the scanner's precision and recall were 1.00 and 1.00 (Wilson lower bounds at 95% confidence: 95.4% and 95.4%). This validation is weaker than it looks. The labels were assigned by an AI assistant (a Claude subagent) that saw the catalogue's definitions but not the scanner, not by an independent human; and the pilot repositories also served to discover signatures, so the sample checks that the scanner implements the catalogue, not that the catalogue covers every tool.

Hypotheses. All shares are pooled over commits, so large repositories weigh more; repository-weighted shares (the mean of repository shares) are reported alongside. Confidence intervals are percentile intervals from 2,000 cluster-bootstrap resamples of repositories within stratum (seed 20261028).

- **H1 (level):** the pooled S1 share in 2026-Q3 is at least 5%.
- **H2 (difference):** S1 exceeds S3 in every month from April to September 2026, and the 2026-Q3 odds ratio is at least two with a CI above one.
- **H3 (growth):** the monthly logit slope of a binomial model is positive in S1 and in S2, and steeper in S1 than in S3 (CIs exclude zero).
- **H4 (recursion):** the 2026-Q3 share in S2 exceeds that in S1 (CI of the difference excludes zero).
- **H5 (tools):** Claude Code has the plurality of attributed commits in S1 ∪ S2 in every quarter from 2025-Q2 to 2026-Q3.

- **H6 (survival):** at least 80% of sampled merged pull requests with an attributed head commit keep the attribution on the default branch.
- **H7 (policy, exploratory):** repositories that require disclosure have a higher share than those that ban AI or are silent.

Holm's correction is applied across H1–H6 for a family-wise claim; each hypothesis is also decided on its own rule, as pre-registered. Analyses not in the pre-registration are labelled *post hoc* (`DEVIATIONS.md`). Pre-registered sensitivity analyses use author dates, include merges, keep only first-parent commits, require trailers in the final trailer block, include maintenance bots, remove the three largest repositories of each stratum, remove mirror-exported repositories, remove outside-caliper matches, and replace the model in H3 with a mixed model with a random intercept and slope per repository.

5 Results

5.1 The curve (RQ1–RQ3)

Figure 1 shows the monthly attributed share. Before 2025 it is close to zero everywhere: in 2024, 0 of 102,709 S1 commits were attributed, 88 of 8,007 in S2 and 5 of 99,830 in S3. The harnesses move first, from late 2024, driven by one project whose agent commits under its own account. S1 and S3 start rising together in mid-2025 and stay close afterwards: over the 2025 calendar year the pooled share was 0.46% in S1 and 0.47% in S3, and over January–September 2026 5.7% and 5.8%.

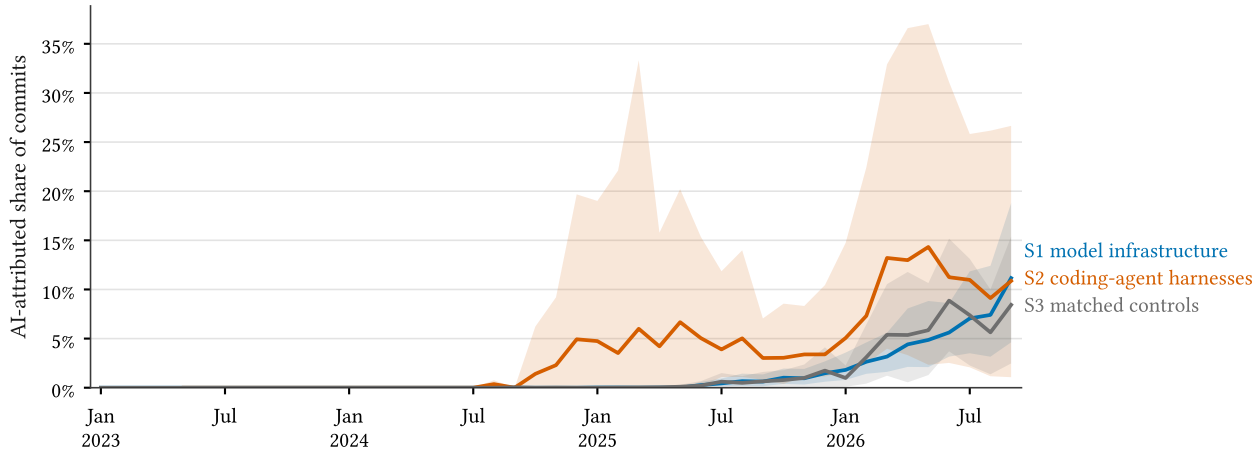


Figure 1: Monthly AI-attributed share of commits by stratum, January 2023 – September 2026, pooled over repositories, with 95% cluster-bootstrap bands. AI-attributed commits are a lower bound on AI involvement.

In 2026-Q3 the pooled share was 8.7% in S1 (3,632 of 41,728 commits; 95% CI 4.0%–14.5%; repository-weighted 10.5%), 10.3% in S2 (1.6%–23.7%; repository-weighted 16.2%) and 7.1% in S3 (2.3%–12.6%; repository-weighted 7.1%). The wide intervals reflect how unevenly attribution is spread across repositories (Figure 6): 39 of the 41 S1 repositories active in the quarter had at least one attributed commit, 15 had a share of ten percent or more, and the median repository share was 4.5%. In S3, 26 of 42 had any attributed commit, and the two largest contributors account for 66% of the stratum's attributed commits; the largest, `home-assistant/core`, alone for 54%. Without it the S3 share is 4.1% (post hoc). Figure 7 in the appendix shows every repository month by month.

Table 1 gives the decisions. **H1 is supported** on its pre-registered rule, which looks at the point estimate: the S1 share in 2026-Q3 is above the 5% floor, but its interval reaches below it, and the one-sided p-value against the threshold is 0.08 (0.41 after Holm's correction). Of H1–H6, only H6 survives the family-wise correction. **H2 is not supported**: S1 was below S3 in 4 of the six pre-registered months (April–September 2026), and the 2026-Q3 odds ratio is 1.24 (95% CI 0.45–4.75). The interval runs from S1 having about half the odds of S3 to almost five times them, so the data neither support the pre-registered doubling nor exclude it. **H3 is not supported**: the monthly logit slope is clearly positive in S1 (0.239) but just as steep in S3 (0.214); the S2 slope (0.078; CI −0.015 to 0.179) includes zero, because the harnesses were already attributing in 2025. **H4 is not supported**: S2 is 1.6 percentage points above S1 in 2026-Q3, with a CI from −9.0 to 15.8 points. Among the harnesses, attribution says more about the tool than about its use: OpenHands' own agent signs most of that repository's commits, while the Codex repository carries almost none.

Table 1: Pre-registered hypotheses. Estimates and 95% CIs from the cluster bootstrap (H6: Wilson interval). p: one-sided p-values against each hypothesis's null: share at or below the threshold (H1), odds ratio at most one (H2), difference at most zero (H3–H5), from the bootstrap; survival share at or below the threshold (H6), exact binomial test. Holm-adjusted across H1–H6. H7 is exploratory.

	Estimate	95% CI	p	p (Holm)	Decision
H1 S1 share, 2026-Q3	8.7%	4.0%–14.5%	0.081	0.407	supported
H2 odds ratio S1 : S3	1.24	0.45–4.75	0.316	1.000	not supported
H3 slope S1 − S3 (logit/month)	0.025	−0.080 to 0.077	0.317	1.000	not supported
H4 share S2 − S1 (points)	1.6	−9.0 to 15.8	0.404	1.000	not supported
H5 Claude Code lead, min. (points)	−75.8	−96.5 to −41.0	1.000	1.000	not supported
H6 attribution survives merge	92.0%	87.4%–95.0%	<0.001	<0.001	supported
H7 disclosure − (ban ∪ silent) (points)	2.9	–	–	–	exploratory

The decisions are the same under every pre-registered sensitivity analysis (Appendix, Table 3): H1 holds and H2–H4 fail under author dates, with merges, on the first-parent chain, with strict trailer positions, with maintenance bots, without the three largest repositories of each stratum, without mirror-exported repositories and without the outside-caliper matches. The estimates, however, move a good deal, which is itself a measure of how fragile they are: the S1 share ranges from 8.1% with strict trailer positions to 12.2% without mirror exports, and the H2 odds ratio from 1.15 to 1.90 without the three largest repositories of each stratum. The mixed model with a random intercept and slope per repository gives the same decision for H3 but a different picture: the S1 − S3 slope difference changes sign, to −0.024 logit per month (Wald CI −0.105 to 0.058), and the S2 slope, 0.122 (0.031 to 0.214), has an interval that excludes zero once each repository has its own trend.

5.2 Which tools (RQ4)

The tool mix changed quickly (Figure 2). In S1 ∪ S2, OpenHands had the plurality of attributed commits through 2025-Q3 (a majority until 2025-Q2), almost all of them in its own repository. Gemini Code Assist led in 2025-Q4 (42%); it is a code-review bot, and its trailer marks a reviewer's suggested change that a contributor applied, as does Copilot's for applied review suggestions. Dropping those two signatures (post hoc) changes the S1 share in 2026-Q3 to 8.5%. Claude Code has led since 2026-Q1, with 34%, 42% and 52% of attributed commits in the three quarters of 2026. **H5 is not supported** because the plurality did not hold in 2025; the minimum margin of Claude Code over the next tool is −75.8 percentage points. Because Claude Code signs by default and many tools do not, its share of *attributed* commits probably overstates its share of AI-assisted commits, by an unknown factor.

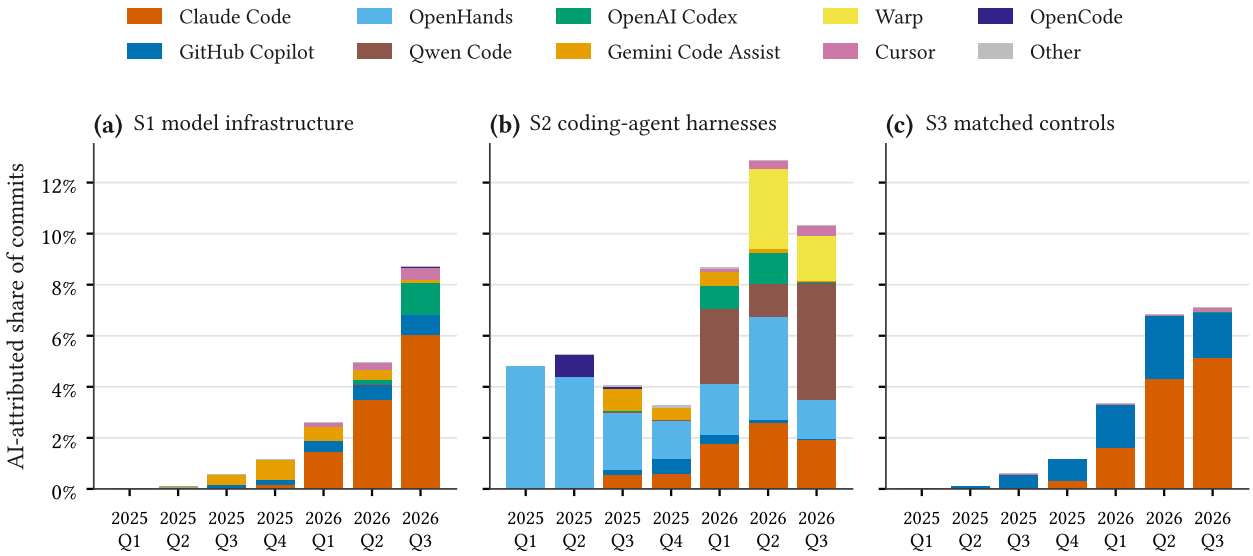


Figure 2: Attributed commits by tool, as a share of all commits in each stratum, by quarter, 2025-Q1 to 2026-Q3. The nine tools with the most attributed commits over 2025–2026 are shown; all others are grouped as Other. AI-attributed commits are a lower bound.

6 How much does the signal miss? (RQ5)

Survival through the merge. The estimand is the share of merged pull requests with at least one attributed commit whose commit on the default branch still carries an attribution, among pull requests merged June 2025 to September 2026 in S1 and S2, each sampled pull request counting once. We sampled them uniformly at random within repository, visiting repositories in turn, and compared the attribution on each pull request's commits with the attribution on what landed. Of 2,542 pull requests inspected, 200 had at least one attributed commit, and in 184 of them an attribution reached the default branch (92.0%, 95% CI 87.4%–95.0%; H6 supported). Figure 3 splits the result by merge method. Only squash merges are an observed test: a merge commit keeps the pull request's commits by construction, and a rebase is scored as kept unless its rewritten last commit lost an attribution that the original had (`DEVIATIONS.md`). Almost all sampled pull requests were squash-merged, and 176 of 192 squash merges kept an attribution, which also clears the pre-registered threshold. The 200 pull requests come from 41 repositories, a few of which contribute many; resampling repositories instead of pull requests gives an interval of 87.3%–96.2% (post hoc), and the losses are spread over 10 repositories rather than concentrated in one. By tool named on the commits, pull requests with Claude Code kept an attribution in 88 of 95, with GitHub Copilot in 18 of 22 and with Warp in 20 of 23.

We cannot say from the landed commit *why* attribution survives a squash. GitHub's documented default message for a multi-commit pull request is its title and a list of its commits [18], which may carry the commits' trailers, and GitHub also credits every commit author as a co-author of the squash commit [19], which keeps an agent account's attribution even when the message is rewritten. A loss can come from a repository's squash setting or from a person editing the message at merge. Pull requests with more than a hundred commits were read only up to their hundredth commit. In the sampled repositories, then, little attribution is lost at the merge; the other paths, trailers switched off, exports from internal repositories and tools that never attribute, are not measured here, and the size of the undercount remains unknown.

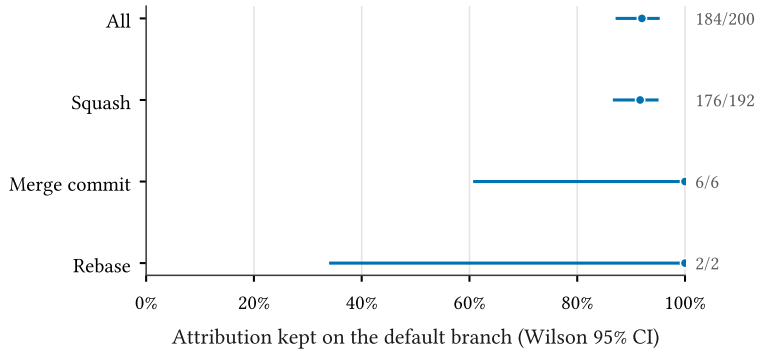


Figure 3: Share of sampled merged pull requests with an attributed commit whose attribution reached the default branch, by merge method, with Wilson 95% intervals.

Policies. Of the 96 repositories, 1 bans AI contributions in its contribution guidelines, 20 require disclosure, 16 mention AI without a rule, and 59 are silent; 55 carry a configuration file addressed to coding agents (`AGENTS.md`, `CLAUDE.md` or Copilot instructions). In 2026-Q3 the pooled share was 9.6% in repositories that require disclosure, 12.0% where AI is mentioned without a rule and 6.8% where policies are silent (Figure 4). The exploratory H7 difference is positive (2.9 percentage points), but it is not the largest contrast: repositories that mention AI without a rule have a higher share than those that require disclosure. Without an interval, and with everything else that distinguishes these repositories, we read nothing causal into it. The policy classes were checked against the guideline text by an AI assistant after the attributed shares had been computed, so the check was not blind.

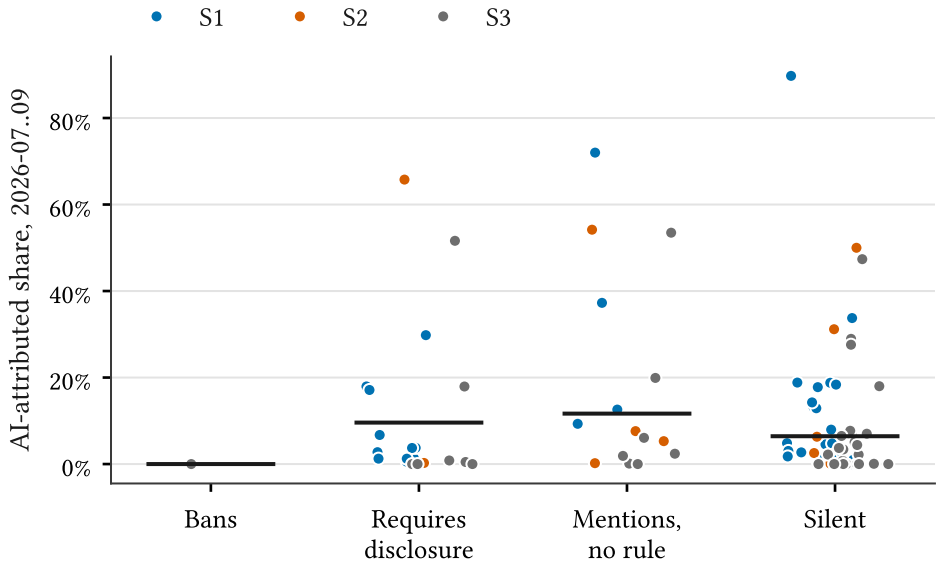


Figure 4: AI-attributed share of commits in 2026-Q3 by the repository's AI-contribution policy (one dot per repository; bars: pooled share of the class). Policies were read at the snapshot from the contribution guidelines, classified by rule and checked by an AI assistant after the shares were known. AI-attributed commits are a lower bound.

Tools that never attribute cannot be measured from git metadata. Classifier-based estimates [9], [11] suggest that unattributed AI involvement is large, but they measure a different quantity, and we make no estimate of it.

7 Exploratory analyses (RQ6)

Size. In a stratified random sample of 2,000 window commits the median attributed commit changed 108 lines in S1 against 32 for unattributed commits; the ratios were similar in S2 and S3 (Appendix, Table 4). Part of this is composition: attributed commits cluster in repositories whose commits are large; part is time, because attributed commits are recent and commit sizes change over the years. Restricted to 2026, the S1 ratio of medians is 1.59 (0.83–3.75). Within repositories and calendar years (fixed effects for both, post hoc), attributed commits were larger by a factor of 1.38 in geometric mean in S1, with an interval that includes one (0.82–2.31), and by 1.78 in S2 and 2.41 in S3. In S1, then, the size difference may be mostly composition and time.

Reverts. The crude revert rate of attributed commits is lower than that of other commits in every stratum, but this too is composition and exposure: attributed commits are recent, so they have had less time to be reverted, and they cluster in some repositories. Compared with unattributed commits of the same repository and month (commits to June 2026; post hoc), attributed commits were reverted 1.17 times as often as expected in S1 (95% CI 0.78–1.68; 29 observed, 24.9 expected). Only reverts made with git's standard message are detected, and the counts are small, so these data cannot say whether attributed commits are reverted more or less often.

Breadth and intensity. Repositories differ in whether attribution is concentrated in a few contributors or spread across many (Figure 5). Repositories above the dashed line have a higher share of attributed commits than of attributing authors, so a few contributors account for most attributed commits; repositories below it have attribution spread thinly over many authors. The line is a visual reference, not a benchmark: the two shares have different denominators.

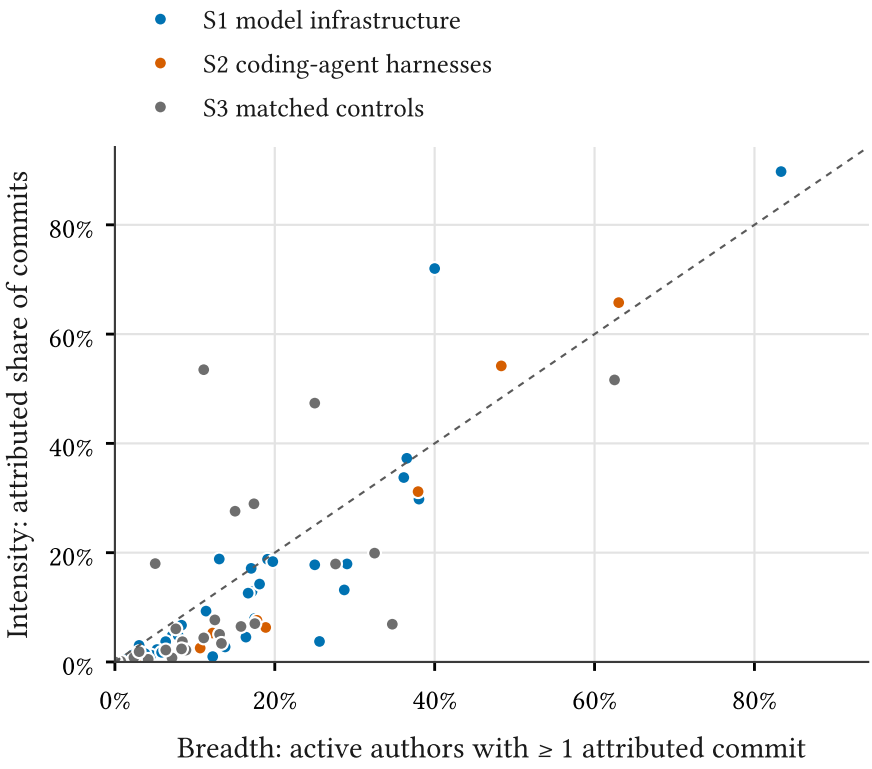


Figure 5: Per repository, 2026-Q3: intensity (share of commits attributed) against breadth (mean monthly share of active authors with at least one attributed commit). Dashed line: equal shares. AI-attributed commits are a lower bound.

8 Discussion

What the curve says about "AI building AI". We had expected AI-infrastructure projects, staffed by people who build models, to attribute earlier and more; in this population the data do not show that. The harnesses, built by the vendors and communities of the tools themselves, attributed earliest, and by 2026 the other two groups had reached similar levels. The comparison is weak in both directions, though: twelve to forty-two repositories per group, one control repository with a large share of its group's attributions, and AI-infrastructure repositories exported from internal monorepos make the intervals wide, and the data are consistent with a moderate lead for either group. A plausible reading, which this design cannot test, is that attribution in mid-2026 depends less on what a project is about than on which tools its contributors use and whether those tools sign by default.

Attribution as a provenance artefact. A co-author trailer is a cheap, machine-readable record of which tool took part in a change, much as a software bill of materials records which components a build includes. In the sampled repositories the record mostly survives the merge. Projects that want to know how their code was produced could keep tool attribution switched on and choose a squash message that keeps commit trailers; whether to ask contributors for that is a governance choice this study does not settle. Neither practice tells anyone how much of a change a tool wrote, and neither should be read as a quality signal.

What the curve does not say. It does not say that one commit in 11 to AI infrastructure was written by AI. An attributed commit may be mostly human work with a small tool contribution, and many AI-assisted commits are not attributed. The curve also does not measure quality, safety or productivity. And it says nothing about model collapse or similar concerns about recursive training [33]; those would require knowing what the attributed code does.

9 Limitations

Lower bound. Attribution undercounts AI involvement by an unknown and probably changing factor; tool defaults, user settings, merge settings and exports from internal repositories all move it. **Attribution is not adoption, nor authorship share.** We count signed commits, not use, lines or effort. **Selection.** S1 is a purposive list of prominent projects; S3 is matched on popularity and activity, not on contributor population, and its two largest contributors carry much of its attributed volume. The results describe these repositories, not open source at large. **Validation.** The labelled sample was drawn from the pilot repositories and labelled by an AI assistant, not an independent human; with no errors in it, the Wilson lower bounds of precision and recall still allow error rates of a few percent. **Small numbers.** Twelve harnesses and forty-two repositories per comparison group give wide intervals, and the pre-registered comparisons have little power against moderate differences. **One snapshot.** Histories were read once, on 2026-10-28; force-pushes and later rewrites are not reflected. **Tool mix.** Default-on signatures dominate the counts, so tool shares describe the instrument as much as the market. **Survival sample frame.** Repositories that land pull requests with a merge bot and close them, rather than merging, contribute few sampled pull requests, and pull requests opened long before the window are outside the frame (`DEVIATIONS.md`). **Policies** were read from the pre-registered files only, and a few repositories state policies elsewhere; the classification was checked by an AI assistant, not blind to the shares. **Survivorship.** Only repositories that are popular and active in 2026 are included; projects that declined or were archived are not.

10 Ethics

The study uses public git metadata and the public GitHub API within its terms. Identities are hashed at ingest with a secret salt; no names, e-mail addresses or commit messages are stored or published, the paper and the dataset report repository-level aggregates only, and no individual developer is named. Repositories are named, because they are public projects and the comparison is between projects; a few live under a personal account, and for those we report nothing beyond what we report for every repository. As pre-registered, only aggregates are released: no per-commit records, and the survival and size samples carry neither pull request numbers nor commit SHAs. The scanner's report command is meant for one's own organisation (`RESPONSIBLE_USE.md`). This is independent personal research using public data only, and it does not represent the views of any employer or institution.

11 Reproducibility

The pre-registration is tagged `prereg-v1` (commit `27a6847`); `DEVIATIONS.md` lists every later clarification. `data/manifest.json` records the head SHA, default branch, commit count and git version of every repository at the snapshot. `coauthored analyze` rebuilds every table, figure and number in this paper from the committed aggregates, with no network and byte-identical output, and `coauthored paper` fills this text from the resulting `results/numbers.json`; the build fails if a number is typed by hand. `coauthored scan` re-derives the aggregates from the recorded SHAs. The code is released under Apache-2.0 and the data under CC BY 4.0.

References

[1] M. Favaro and J. Clark, "When AI builds itself." Anthropic Institute, 2026. Available: https://www.anthropic.com/institute/recursive-self-improvement

[2] Anthropic, "All settings – Claude Code docs (`attribution`, `includeCoAuthoredBy`)." Claude Code documentation, 2026. Available: https://code.claude.com/docs/en/settings-reference

[3] GitHub, "Risks and mitigations for GitHub Copilot cloud agent." GitHub Docs, 2026. Available: https://docs.github.com/en/copilot/concepts/agents/coding-agent/risks-and-mitigations

[4] Anysphere (Cursor), "Secrets & network (Cloud Agents); Git integration." Cursor documentation, 2026. Available: https://cursor.com/docs/cloud-agent/security-network

[5] Google, "Auto-fixing CI failures and configure Jules to commit as you." Jules changelog, Feb. 2026. Available: https://jules.google/docs/changelog/2026-02-19

[6] Aider, “Git integration.” Aider documentation, 2026. Available: https://aider.chat/docs/git.html

[7] OpenAI, “Codex: `codex-rs/ext/git-attribution` (source code).” GitHub repository openai/codex, 2026. Available: https://github.com/openai/codex/blob/main/codex-rs/ext/git-attribution/src/world_state.rs

[8] R. Robbes, T. Matricon, T. Degueule, A. Hora, and S. Zacchiroli, “Agentic much? Adoption of coding agents on GitHub,” *ACM Transactions on Software Engineering and Methodology*, 2026, doi: 10.1145/3822180.

[9] K. Horikawa *et al.*, “Agent or human? Detecting commits by AI coding agents in the absence of authorship signatures.” 2026. doi: 10.48550/arXiv.2610.36229.

[10] P. Kinlan, “Damn claude, that’s a lot of commits.” aifoc.us (blog), post #024, Mar. 2026. Available: https://aifoc.us/damn-claude-thats-a-lot-of-commits/

[11] S. Daniotti, J. Wachs, X. Feng, and F. Neffke, “Who is using AI to code? Global diffusion and impact of generative AI,” *Science*, vol. 391, no. 6787, pp. 831–835, 2026, doi: 10.1126/science.adz9311.

[12] H. Li, H. Zhang, and A. E. Hassan, “AIDev: Studying AI coding agents on GitHub,” in *Proceedings of the 23rd international conference on mining software repositories (MSR ‘26)*, 2026, pp. 1029–1033. doi: 10.1145/3793302.3797249.

[13] D. O’Laughlin, J. Eliahou Ontiveros, J. Nanos, *et al.*, “Claude code is the inflection point.” SemiAnalysis newsletter, Feb. 2026. Available: https://newsletter.semianalysis.com/p/claude-code-is-the-inflection-point

[14] Amplifying, “Claude code – coding agents index.” Web dashboard, 2026. Available: https://amplifying.ai/coding-agents/claude-code

[15] GitHub, “Octoverse: Commits, pushes, reach new highs in 2026 as developers build with agents.” GitHub Blog, Oct. 2026. Available: https://github.blog/news-insights/octoverse/octoverse-commits-pushes-reach-new-highs-in-2026-as-developers-build-with-agents/

[16] T. Dey *et al.*, “Detecting and characterizing bots that commit code,” in *Proceedings of the 17th international conference on mining software repositories (MSR ‘20)*, 2020, pp. 209–219. doi: 10.1145/3379597.3387478.

[17] GitHub, “Creating a commit with multiple authors.” GitHub Docs, 2026. Available: https://docs.github.com/en/pull-requests/committing-changes-to-your-project/creating-and-editing-commits/creating-a-commit-with-multiple-authors

[18] GitHub, “Configuring commit squashing for pull requests.” GitHub Docs, 2026. Available: https://docs.github.com/en/repositories/configuring-branches-and-merges-in-your-repository/configuring-pull-request-merges/configuring-commit-squashing-for-pull-requests

[19] GitHub, “Improved attribution when squashing commits.” GitHub Changelog, Dec. 2019. Available: https://github.blog/changelog/2019-12-19-improved-attribution-when-squashing-commits/

[20] Gentoo Council, “Project:council/AI policy.” Gentoo Wiki, 2024. Available: https://wiki.gentoo.org/wiki/Project:Council/AI_policy

[21] The NetBSD Foundation, “NetBSD commit guidelines.” netbsd.org, 2024. Available: https://www.netbsd.org/developers/commit-guidelines.html

[22] QEMU Project, “Code provenance.” QEMU developer documentation, 2025. Available: https://www.qemu.org/docs/master/devel/code-provenance.html

[23] curl project, “Contributing to the curl project: On AI use in curl.” curl.se, 2025. Available: https://curl.se/dev/contribute.html

[24] Ghostty project, “AI usage policy (`AI_POLICY.md`).” GitHub repository ghostty-org/ghostty, 2026. Available: https://github.com/ghostty-org/ghostty/blob/main/AI_POLICY.md

[25] The Linux Kernel Documentation, “AI coding assistants (`Documentation/process/coding-assistants.rst`).” docs.kernel.org, 2026. Available: https://docs.kernel.org/process/coding-assistants.html

[26] A. Hora and R. Robbes, “AI policy, disclosure, and human in the loop: How are contribution guidelines adapting to GenAI?” in *Proceedings of the IEEE international conference on software maintenance and evolution (ICSME 2026)*, 2026. doi: 10.48550/arXiv.2605.16706.

[27] S. Peng, E. Kalliamvakou, P. Cihon, and M. Demirer, "The impact of AI on developer productivity: Evidence from GitHub Copilot." 2023. doi: 10.48550/arXiv.2302.06590.

[28] K. Z. Cui, M. Demirer, S. Jaffe, L. Musolff, S. Peng, and T. Salz, "The effects of generative AI on high-skilled work: Evidence from three field experiments with software developers," *Management Science*, 2026, doi: 10.1287/mnsc.2025.00535.

[29] J. Becker, N. Rush, E. Barnes, and D. Rein, "Measuring the impact of early-2025 AI on experienced open-source developer productivity." 2025. doi: 10.48550/arXiv.2507.09089.

[30] D. Yeverechyahu, R. Mayya, and G. Oestreicher-Singer, "The impact of large language models on open-source innovation: Evidence from GitHub Copilot." 2024. doi: 10.48550/arXiv.2409.08379.

[31] Anthropic, "Anthropic economic index: AI's impact on software development." Anthropic research, Apr. 2025. Available: https://www.anthropic.com/research/impact-software-development

[32] T. Kwa *et al.*, "Measuring AI ability to complete long software tasks." 2025. doi: 10.48550/arXiv.2503.14499.

[33] I. Shumailov, Z. Shumaylov, Y. Zhao, N. Papernot, R. Anderson, and Y. Gal, "AI models collapse when trained on recursively generated data," *Nature*, vol. 631, no. 8022, pp. 755–759, 2024, doi: 10.1038/s41586-024-07566-y.

Appendix

Repositories

Table 2: Study repositories by stratum and S1 layer, at the 2026-10-28 snapshot. S3 controls are grouped by the layer of the S1 repository each was matched to; the pairs are in `data/s3_match.json`. Exclusions and their reasons: `configs/repos.yaml`, `configs/s3_screen.yaml`.

Group	Repositories
S1 · Training frameworks	google/flax, jax-ml/jax, keras-team/keras, pytorch/pytorch, tensorflow/tensorflow
S1 · Model libraries	huggingface/accelerate, huggingface/datasets, huggingface/diffusers, huggingface/peft, huggingface/tokenizers, huggingface/transformers, huggingface/trl
S1 · Distributed training	Lightning-AI/pytorch-lightning, NVIDIA-NeMo/Speech, NVIDIA/Megatron-LM, deepspeedai/DeepSpeed, pytorch/torchtitan
S1 · Fine-tuning and RL	allenai/open-instruct, axolotl-ai-cloud/axolotl, hiyouga/LlamaFactory, unslothai/unsloth
S1 · Inference and serving	InternLM/lmdeploy, ggml-org/ggml, ggml-org/llama.cpp, microsoft/onnxruntime, ml-explore/mlx, ml-explore/mlx-lm, modular/modular, ollama/ollama, pytorch/executorch, sgl-project/sglang, vllm-project/vllm
S1 · Kernels and compilers	Dao-AILab/flash-attention, NVIDIA/TransformerEngine, bitsandbytes-foundation/bitsandbytes, flashinfer-ai/flashinfer, openxla/xla, pytorch/ao, triton-lang/triton
S1 · Evaluation and data	EleutherAI/lm-evaluation-harness, huggingface/datatrove, stanford-crfm/helm
S2 · Coding-agent harnesses	Kilo-Org/kilocode, OpenHands/OpenHands, QwenLM/qwen-code, aaif-goose/goose, anomalyco/opencode, charmbracelet/crush, cline/cline, continuedev/continue, google-gemini/gemini-cli, openai/codex, openinterpreter/openinterpreter, warpdotdev/warp
S3 · controls for training frameworks	conan-io/conan, django/django, frappe/erpnext, godotengine/godot, home-assistant/core
S3 · controls for model libraries	aio-libs/aiohttp, bridgecrewio/checkov, fastapi/fastapi, grpc/grpc-rust, nicolargo/glances, saleor/saleor, tornadoweb/tornado
S3 · controls for distributed training	enjoy-digital/litex, fastapi/sqlmodel, kovidgoyal/calibre, python-poetry/poetry, soxoj/maigret
S3 · controls for fine-tuning and RL	WebODM/WebODM, pwndbg/pwndbg, python/cpython, sherlock-project/sherlock
S3 · controls for inference and serving	RPCS3/rpcs3, argotorg/solidity, bitcoin/bitcoin, ccxt/ccxt, eclipse-sumo/sumo, microsoft/TypeScript, mongodb/mongo, moraroy/NonSteamLaunchers-On-Steam-Deck, odoo/odoo, organicmaps/organicmaps, saulpw/visidata
S3 · controls for kernels and compilers	GAM-team/GAM, LibreOffice/core, libsdl-org/SDL, locustio/locust, pgadmin-org/pgadmin4, qutebrowser/qutebrowser, wesnoth/wesnoth
S3 · controls for evaluation and data	ansible/ansible-lint, mikumifa/biliTickerBuy, redis/redis-py

Sensitivity analyses

Table 3: Estimates under each sensitivity analysis (✓ supported, ✗ not supported); rows marked post hoc were not pre-registered. H1: S1 share, 2026-Q3; H2: odds ratio S1 : S3; H3: slope difference S1 − S3; H4: share difference S2 − S1. The GLMM row replaces the model in H3.

analysis	H1	H2	H3	H4
primary	0.087 ✓	1.243 ✗	0.025 ✗	0.016 ✗
author_date	0.087 ✓	1.189 ✗	0.020 ✗	0.016 ✗
bots_included	0.087 ✓	1.433 ✗	0.031 ✗	0.023 ✗
first_parent	0.091 ✓	1.268 ✗	0.020 ✗	0.123 ✗
merges_included	0.085 ✓	1.337 ✗	0.024 ✗	0.008 ✗
no_review_suggestions (post hoc)	0.085 ✓	1.498 ✗	0.065 ✗	0.018 ✗
strict_trailers	0.081 ✓	1.149 ✗	0.021 ✗	0.022 ✗
largest_3_removed	0.109 ✓	1.899 ✗	−0.027 ✗	0.063 ✗
mirror_removed	0.122 ✓	1.808 ✗	0.017 ✗	−0.019 ✗
outside_caliper_removed	0.087 ✓	1.267 ✗	0.013 ✗	0.016 ✗
glmm			−0.024 ✗	

Commit size

Table 4: Lines changed (insertions plus deletions) in the size sample: median [interquartile range] for attributed and unattributed commits, and the ratio of medians with a bootstrap 95% CI. Rows labelled 2026 restrict both groups to January–September 2026.

period	stratum	n_attr	n_unattr	attributed	unattributed	ratio (95% CI)
all	S1	334	334	108 [22, 359]	32 [7, 130]	3.38 [1.82, 5.06]
all	S2	333	333	155 [42, 665]	52 [16, 218]	2.98 [2.00, 4.50]
all	S3	333	333	64 [20, 193]	20 [6, 84]	3.20 [2.33, 5.24]
2026	S1	298	88	114 [22, 394]	72 [8, 230]	1.59 [0.83, 3.75]
2026	S2	256	186	220 [45, 932]	62 [21, 260]	3.51 [1.90, 5.70]
2026	S3	308	85	64 [21, 196]	16 [4, 84]	4.03 [2.12, 8.12]

Within repository, post hoc (ratio of geometric means of 1 + lines, repository and calendar-year fixed effects, CI clustered by repository):

stratum	repos	n	ratio (95% CI)
S1	23	589	1.38 [0.82, 2.31]
S2	9	639	1.78 [1.33, 2.37]
S3	14	525	2.41 [1.21, 4.81]

Per-repository shares



Figure 6: AI-attributed share of commits per repository, 2026-Q3, with Wilson 95% intervals. One S1 repository had no commits in the quarter. AI-attributed commits are a lower bound.

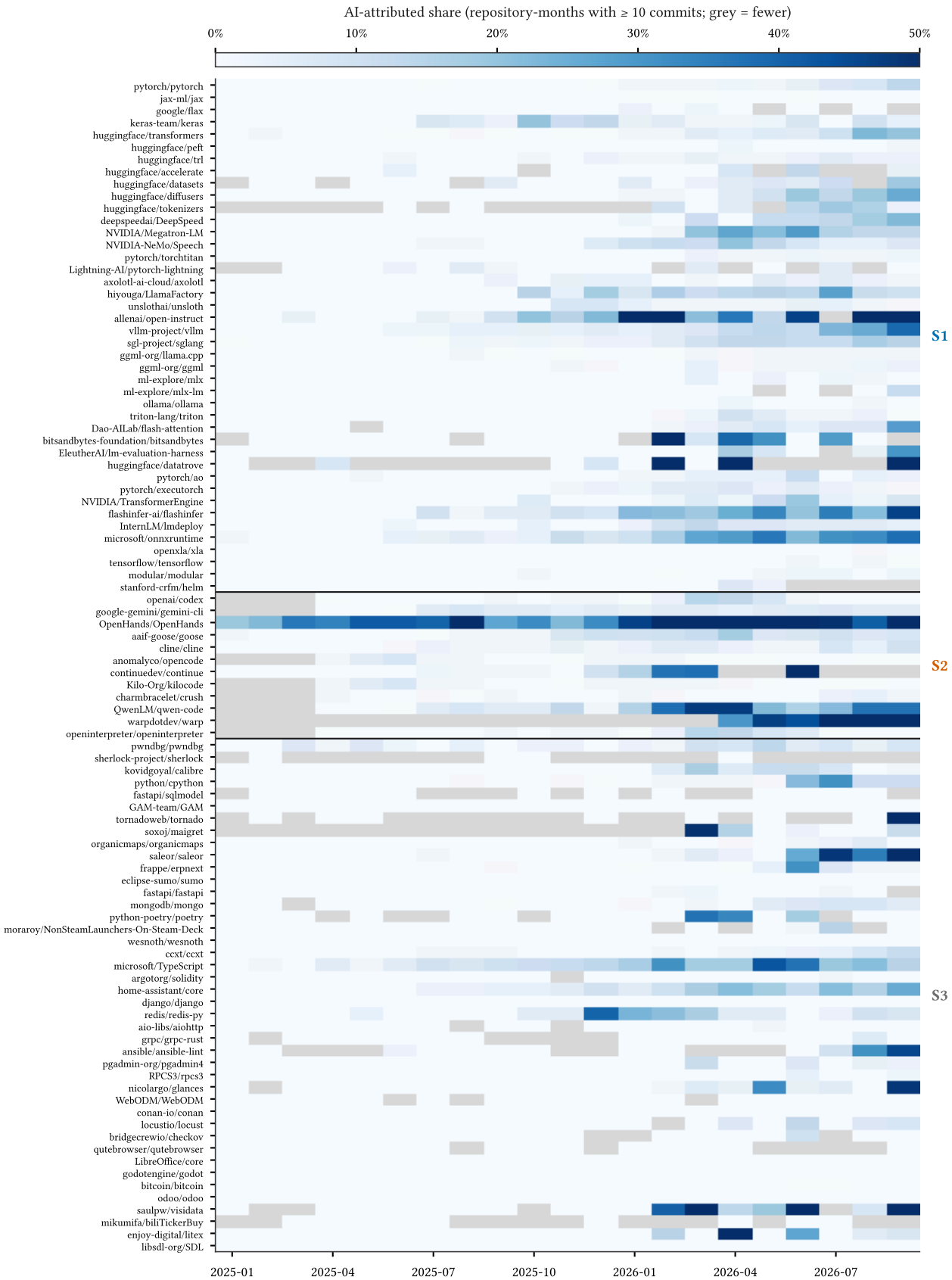


Figure 7: AI-attributed share per repository and month, 2025-01 to 2026-09; repository-months with fewer than ten commits in grey. AI-attributed commits are a lower bound.